SIMATS ENGINEERING
ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I NANDHINI SRI V(192521344) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence

Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Reverse Engineering of a Big Data Platform

A big data platform that collects billions of website clicks, mobile interactions and customer transactions needs a distributed architecture because a single system cannot handle such a large amount of data.

Probable Architecture

The likely system flow is:

Website / Mobile Apps / Transactions → Data Ingestion → Distributed Storage → Data Processing → Stream Processing → Analytics → Dashboard

1. Distributed Storage

The platform may use **Hadoop Distributed File System (HDFS)** or cloud object storage to store the huge amount of data.

Structured data such as customer and transaction records can be stored in databases. Semi-structured data such as click logs and mobile events can be stored in a data lake.

2. Data Ingestion

Tools such as **Apache Kafka** are likely used to collect website clicks, mobile interactions and transactions continuously.

Kafka is suitable because it can handle a large number of events and send them to different processing systems.

3. Batch Processing

Apache Spark can be used for batch processing. It can process large amounts of stored data and find useful patterns.

For example, it can analyze customer purchases and website activity to understand customer behavior.

4. Stream Processing

For real-time analytics, technologies such as **Spark Streaming or Apache Flink** can be used. They process incoming data immediately.

For example, when a customer clicks on a product, the event can be analyzed in real time.

5. Data Separation and Indexing

Structured data such as customer details and transactions can be stored in relational or structured databases.

Semi-structured data such as JSON clickstream logs can be stored in a data lake. Indexing can be used to quickly search and retrieve important information.

6. Scalability and Partitioning

Since the platform handles billions of records, the data is likely divided into smaller partitions. Data can be partitioned based on **date, customer, region or event type**.

Multiple servers can process different partitions at the same time. This improves processing speed and allows the system to handle increasing data.

7. Dashboard and Business Intelligence

The processed data can be connected to dashboard and business intelligence tools. These dashboards can show information such as:

- Number of website visitors
- Customer purchases
- Popular products
- Sales trends
- Customer behavior

Managers can use these dashboards for quick decision-making.

Conclusion

From the given description, the platform is likely built using **distributed storage, Kafka for data ingestion, Spark for batch processing and streaming tools for real-time analytics**. Partitioning and multiple servers help the system handle billions of records. Finally, dashboards use the processed data to provide useful business information.

2.An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Reverse Engineering of an E-Commerce Big Data System

An e-commerce platform collects different types of customer activities such as product views, cart additions, abandoned checkouts and payments. From this information, we can understand that the platform probably uses a combination of **streaming, databases, caching, big data processing and machine learning**.

Probable System Design

The possible workflow is:

Customer Activity → Event Streaming → Data Storage → Data Processing → Customer Profile → Recommendation → Dashboard

Event Streaming

Tools like **Apache Kafka** are likely used to collect customer activities in real time. Each activity, such as viewing a product or adding it to the cart, can be treated as an event.

For example, when a customer adds a mobile phone to the cart, the event can immediately reach the processing system.

Customer Profile Database

A customer profile database stores information about customers, including their previous purchases, interests and activities.

The system can update the customer profile whenever a new activity occurs.

Real-Time Personalization

Session data, clickstream data and transaction data are combined to understand the customer's current interest.

For example, if a customer is viewing shoes and has previously purchased sportswear, the system can immediately recommend sports shoes.

Caching System

A caching system such as **Redis** can store frequently required information. This reduces the time required to retrieve product and customer information.

This is useful when thousands of customers are using the website at the same time.

Distributed Query and Processing

A distributed query engine can process data stored across multiple systems. Technologies such as **Apache Spark** can process large datasets and identify customer behavior patterns.

Machine Learning

Machine learning can be used to analyze customer behavior and generate product recommendations.

The process may be:

Customer Data → Data Processing → Pattern Identification → Recommendation Model → Product Recommendation

Combining Different Data

Structured data such as order details and payment records can be combined with semi-structured data such as browsing logs and clickstream information.

This gives the company a complete view of customer activity.

Scalability and Fault Tolerance

The platform can use multiple servers to distribute the workload. Data can also be replicated so that another copy is available if one server fails.

This allows the system to continue working even when a component has a problem.

KPI Dashboard

The processed information can be connected to dashboards to monitor important KPIs such as:

- Total sales
- Number of orders
- Abandoned carts
- Most viewed products
- Customer activity
- Conversion rate

Conclusion

From the given system, it can be inferred that the e-commerce platform probably uses **Kafka for event streaming, Redis for caching, distributed processing for large datasets and machine learning for recommendations**. Combining browsing and transaction data helps provide real-time personalization and better business decisions.

3.A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Reverse Engineering of a Big Data Healthcare System

The given healthcare system handles different types of information such as patient history, diagnostic images, wearable sensor data and prescription records. Since these data types are different, the system probably uses a **hybrid architecture** with different storage and processing methods.

Probable Architecture

Hospitals / Labs / Wearable Devices → Data Integration → Hybrid Storage → Processing → Analytics → Healthcare Decision

1. Hybrid Database Architecture

Structured information such as patient details, prescriptions and medical records can be stored in a relational database.

Unstructured information such as X-rays, CT scans and medical images can be stored in cloud or distributed file storage.

Therefore, using different storage systems is suitable for managing different types of healthcare data.

2. Data Integration

Hospitals, laboratories and insurance systems generate data in different formats. Data integration tools can collect and combine this information into one platform.

Standards such as **HL7 and FHIR** can help different healthcare systems exchange information.

3. Streaming Data

Wearable devices continuously produce data such as heart rate, temperature and activity levels.

A streaming system such as **Apache Kafka** can collect this information continuously. Real-time processing can help identify unusual changes in a patient's condition.

4. Machine Learning

Machine learning can be used to study patient information and identify possible health risks.

For example, the system can analyze patient history and sensor readings to identify patterns that may indicate a disease.

5. Privacy and Security

Healthcare data is sensitive, so strong security is required. The system can use:

- Data encryption
- User authentication
- Access control
- Secure data storage
- Activity monitoring

Only authorized healthcare staff should be allowed to access patient information.

6. Analytics Pipeline

The healthcare analytics process can be:

Data Collection → Data Cleaning → Data Integration → Data Processing → Analysis → Medical Decision

The processed information can help doctors understand patient conditions and support clinical decisions.

7. Population Health Analysis

The system can also analyze data from a large number of patients. This can help identify common diseases, health trends and risk factors in a population.

Example

If wearable devices show that a patient's heart rate is continuously abnormal, the system can analyze the readings and patient history. It can then alert healthcare professionals for further examination.

Conclusion

The healthcare platform probably uses a **hybrid database, data integration tools, streaming technologies and machine learning**. Security measures such as encryption and access control protect patient information. The complete system can support real-time monitoring, disease prediction and better healthcare decisions.

4.A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Reverse Engineering of a Smart City Big Data Platform

A smart city platform collects information continuously from **traffic sensors, CCTV systems, weather stations and public transport systems**. Since the data is generated from many sources and changes continuously, the platform needs both real-time and batch processing.

Probable System Architecture

The likely architecture can be represented as:

Sensors / CCTV / Weather / Transport → Edge Computing → Message System → Data Storage → Real-Time Processing → Analytics → Visualization

1. Edge Computing

Edge computing is probably used near the data source. It processes some data before sending it to the central system.

For example, traffic sensors can identify the number of vehicles on a road locally and send only useful information to the main system.

This reduces network traffic and improves response time.

2. Distributed Messaging System

A system such as **Apache Kafka** can be used to collect continuous data from different sources.

Traffic sensors, public transport systems and weather stations can send their information as events. Kafka can handle a large number of events at the same time.

3. Centralized Storage

The collected data can be stored in a cloud-based data lake or distributed storage system.

Different types of information can be stored, such as:

- Traffic sensor data
- Weather information
- Public transport records
- CCTV metadata
- Location information

A distributed storage system is useful because the amount of smart city data can become very large.

4. Real-Time Data Processing

Real-time processing tools such as **Apache Spark Streaming or Apache Flink** can analyze incoming data immediately.

For example, if traffic sensors show that the number of vehicles is increasing rapidly, the system can detect possible congestion.

5. Geospatial Analytics

Smart city data is closely related to locations. Geospatial analytics can be used to understand where traffic congestion or other problems are occurring.

For example, the system can identify which roads have heavy traffic and find nearby alternative routes.

6. Congestion Prediction

Historical traffic data can be combined with current traffic information to predict congestion.

The system can consider:

- Number of vehicles
- Time of day
- Weather conditions
- Previous traffic patterns
- Road location

This helps city authorities take action before the traffic problem becomes serious.

7. Time-Series Database

Traffic and sensor data is continuously generated with timestamps. A **time-series database** can be used to store this type of information efficiently.

It helps analyze how traffic conditions change over time.

8. Location-Aware Database

A location-aware database can store information based on geographical locations.

For example, it can help find all traffic sensors or buses located within a particular area.

9. Batch Analytics

Not all data needs to be processed immediately. Historical data can be processed in batches to identify long-term trends.

For example, the city can analyze several months of traffic data to identify roads that regularly experience congestion.

10. Combining Real-Time and Batch Analytics

Both methods can work together.

Real-Time Data → Immediate Decisions

Historical Data → Long-Term Planning

For example, real-time analytics can identify a traffic jam today, while batch analytics can show that the same road has traffic problems every evening.

11. Security

Smart city systems contain important infrastructure and citizen-related information. Therefore, security measures are required.

The system can use:

- User authentication
- Role-based access control
- Data encryption
- Secure communication
- Regular security monitoring
- Network protection

Only authorized users should be able to access important city data.

12. Visualization

The analyzed information can be displayed using dashboards and maps.

City authorities can view:

- Current traffic conditions
- Traffic congestion areas
- Bus locations
- Weather conditions
- Sensor status
- Emergency situations

This provides live operational information and helps authorities respond quickly.

Example

Suppose traffic sensors detect a sudden increase in vehicles on a particular road. The streaming system processes the information immediately and identifies congestion. The dashboard displays the affected location on a map. City officials can then control traffic signals or suggest alternative routes.

At the same time, the historical data can be analyzed later to understand why congestion frequently occurs at that location.

Conclusion

From the given description, the smart city platform probably uses **edge computing, Kafka, distributed storage, streaming processing, geospatial analytics and time-series databases**. Real-time analytics helps in immediate traffic management, while batch analytics supports long-term urban planning. Security

and access control protect important city and citizen data, and dashboards provide live information for better decision-making.

5.A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations**.**

Reverse Engineering of a Healthcare Analytics System

The given healthcare analytics system collects different types of data such as **patient records, wearable device readings, lab reports and appointment histories**. Since the data is large, sensitive and available in different formats, the system probably uses a **hybrid storage and processing architecture**.

Probable System Architecture

Patient Data → Data Collection → ETL & Integration → Hybrid Storage → Distributed Processing → Real-Time Monitoring → Machine Learning → Healthcare Decision

1. Hybrid Storage Architecture

The system probably uses different storage systems for different types of data.

Structured data such as patient details, appointment information and lab results can be stored in a **relational database**.

Unstructured or semi-structured data such as medical reports and wearable device data can be stored in **cloud storage or a data lake**.

This makes it easier to manage different types of healthcare information.

2. ETL Pipeline

ETL means **Extract, Transform and Load**.

- **Extract** – Data is collected from hospitals, clinics, laboratories and wearable devices.
- **Transform** – The data is cleaned and converted into a common format.
- **Load** – The processed data is stored in the required database or data lake.

This process helps combine data from different healthcare systems.

3. Interoperability Standards

Hospitals and clinics may use different systems to store patient information. Standards such as **HL7 and FHIR** can help exchange healthcare information between different systems.

This allows patient information to be shared in a standard and organized way.

4. Real-Time Monitoring

Wearable devices can continuously collect information such as:

- Heart rate
- Blood pressure
- Body temperature
- Physical activity
- Sleep information

This data can be processed in real time. If an unusual change is detected, the system can generate an alert.

5. Predictive Health Analytics

The system can use previous patient information to predict possible health risks.

For example, patient history, lab results and wearable readings can be analyzed together to identify whether a patient has a higher risk of developing a particular health problem.

6. Distributed Processing

Healthcare organizations may have data from thousands or millions of patients. Processing all this data using a single computer would take more time.

Distributed processing frameworks such as **Apache Spark** can divide the workload among multiple computers.

This helps perform population-level analysis faster.

7. Population-Level Analysis

The system can analyze information from a large number of patients to identify:

- Common diseases
- Health trends
- Risk factors
- Treatment patterns
- Changes in disease rates

This information can help healthcare organizations plan better services.

8. Encryption

Healthcare data is sensitive and must be protected.

Encryption can be used while storing and transferring data. This prevents unauthorized people from easily reading the information.

9. Access Control

Not every employee should have access to all patient information.

Role-based access control can be used. For example, doctors may access patient medical records, while administrative staff may only access appointment information.

10. Audit Mechanism

The system should maintain records of who accessed or changed patient information.

Regular auditing helps identify unauthorized access and supports proper data governance.

11. Machine Learning

Machine learning models can analyze patient information and provide risk scores.

For example, a model can analyze age, medical history, lab results and other information to estimate the risk of a disease.

The result can support doctors in making better decisions. However, the final medical decision should be made by qualified healthcare professionals.

Example

Suppose a patient uses a wearable device that continuously records heart rate. The system receives this data in real time and compares it with the patient's previous records and lab results.

If the system detects an unusual pattern, it can generate an alert. A machine learning model can also calculate the patient's risk level, which can help the doctor decide whether further examination is required.

Benefits

- Combines healthcare data from different sources
- Supports real-time patient monitoring
- Helps identify health risks
- Processes large amounts of patient data
- Supports population health studies
- Improves data security
- Helps doctors in decision-making

Conclusion

From the given description, the healthcare analytics system probably uses **hybrid storage, ETL pipelines, HL7/FHIR standards, distributed processing and machine learning**. Encryption, access control and auditing help protect sensitive healthcare information. The system can support real-time monitoring, population-level analysis and risk prediction while helping healthcare professionals make better decisions.